

Typeform Clone: Codebase & Backend Audit Report

Architectural Inspection, Database Verification & Implementation Gap Analysis

REPOSITORY

Typeform-Clone_Scalar-Ai

FRAMEWORK / ORM

FastAPI 0.141 / SQLAlchemy 2.1

DATABASE

SQLite (backend/forms.db)

AUDIT DATE

September 25, 2026

Executive Summary

The existing backend adheres strictly to the prescribed 5-tier architecture: `Route` → `Service` → `Repository` → `SQLAlchemy ORM` → `SQLite`. The core form management flow (Create, Read, List, Update title, Delete) and collision-resistant slug generation are operational. However, the repository contains **no question models or endpoints, no response submission or storage logic, no publish/unpublish workflow, no analytics aggregation, and no frontend application.**

A. Current Architecture

`Route` → `Service` → `Repository` → `SQLAlchemy ORM` → `SQLite`

LAYER / SUBSYSTEM	SOURCE FILE(S)	DESIGN ASSESSMENT & MODULARITY
Route Layer	<code>app/routes/forms.py</code>	Uses FastAPI <code>APIRouter</code> . Zero direct DB queries ; strictly calls <code>FormService</code> injected via <code>Depends(get_form_service)</code> .
Service Layer	<code>app/services/form_service.py</code>	Instantiates <code>FormRepository(db)</code> . Owns business logic: slugification, uniqueness enforcement (<code>_unique_slug</code>), and domain exceptions.
Repository Layer	<code>app/repositories/form_repository.py</code>	Encapsulates all SQLAlchemy queries: <code>create</code> , <code>get_by_id</code> , <code>get_by_slug</code> , <code>get_all</code> , <code>update</code> , <code>delete</code> . Clean CRUD abstraction.
ORM & Models	<code>app/models/form.py</code> <code>app/database/base.py</code>	Modern SQLAlchemy 2.0 <code>DeclarativeBase</code> with typed <code>Mapped[...]</code> and <code>mapped_column(...)</code> . Clean column constraints.
Session Management	<code>app/database/connection.py</code>	<code>SessionLocal = sessionmaker(...)</code> . <code>get_db()</code> generator dependency yields session and guarantees <code>db.close()</code> in <code>finally</code> block.
Validation & Errors	<code>app/schemas/form.py</code> <code>app/core/exceptions.py</code>	Pydantic v2 schemas (<code>FormCreate</code> , <code>FormUpdate</code> , <code>FormRead</code>). Custom <code>AppException</code> caught by global exception handler in <code>main.py</code> .
CORS & Settings	<code>app/core/config.py</code> <code>app/main.py</code>	<code>CORSMiddleware</code> configured with allowed origins for <code>localhost:3000</code> and <code>127.0.0.1:3000</code> . Pydantic <code>BaseSettings</code> with <code>.env</code> support.
Runtime Status	FastAPI Lifespan	Verified active and operational. Imports cleanly; ASGI tests to <code>/health</code> and <code>/api/forms</code> return HTTP 200 OK.

B. Current Database Schema

The SQLite database (`backend/forms.db`) currently contains exactly one active table: `forms` .

COLUMN	SQL TYPE	NULLABLE	DEFAULTS / CONSTRAINTS	DESCRIPTION & USAGE
<code>id</code>	<code>INTEGER</code>	No	<code>PRIMARY KEY AUTOINCREMENT</code>	Unique auto-incrementing form identifier
<code>title</code>	<code>VARCHAR(255)</code>	No	None (Validated 1-255 chars)	Display title of the form
<code>slug</code>	<code>VARCHAR(255)</code>	No	<code>UNIQUE, INDEXED</code>	URL-safe slug for shareable access
<code>status</code>	<code>VARCHAR(50)</code>	No	<code>'draft'</code>	Form status lifecycle state (<code>draft</code> , <code>published</code>)
<code>created_at</code>	<code>DATETIME</code>	No	<code>CURRENT_TIMESTAMP / utc_now</code>	UTC creation timestamp
<code>updated_at</code>	<code>DATETIME</code>	No	<code>CURRENT_TIMESTAMP / onupdate</code>	UTC last modification timestamp

Schema Deficit Notice

MISSING TABLES

The database currently lacks models and tables for `questions` (with question types, settings, and display ordering), `question_options` (for multiple choice and dropdown selections), `responses` (submission metadata), and `answers` (individual question responses).

C. Current API Endpoints

All currently registered endpoints in `backend/app/main.py` and `backend/app/routes/forms.py` :

METHOD	ROUTE PATH	PAYLOAD BODY	HTTP STATUS	DESCRIPTION
<code>GET</code>	<code>/health</code>	None	<code>200 OK</code>	Service health check returning <code>{"status": "ok"}</code>
<code>POST</code>	<code>/api/forms</code>	<code>{"title": str}</code>	<code>201 Created</code>	Creates form, generates unique slug, sets status to <code>draft</code>
<code>GET</code>	<code>/api/forms</code>	None	<code>200 OK</code>	Returns list of all forms ordered by <code>created_at DESC</code>
<code>GET</code>	<code>/api/forms/{id}</code>	None	<code>200 / 404</code>	Retrieves a single form by ID (raises <code>NotFoundError</code> if absent)
<code>PATCH</code>	<code>/api/forms/{id}</code>	<code>{"title": str}</code>	<code>200 / 404</code>	Updates form title by ID
<code>DELETE</code>	<code>/api/forms/{id}</code>	None	<code>204 No Content</code>	Deletes form record by ID

D & E. Feature Comparison Matrix

Audit of existing functionality against core SDE assignment requirements:

REQUIREMENT	CURRENT STATUS	AUDIT FINDINGS & IMPLEMENTATION GAP
Form CRUD & Title	IMPLEMENTED	Create, read, list, rename title, and delete form endpoints are operational.
Publish / Unpublish	MISSING	<code>status</code> column exists in DB, but cannot be updated via <code>FormUpdate</code> or dedicated API routes.
Ordered Questions	MISSING	No <code>Question</code> model, schema, repository, or service exists.
8 Question Types short text, long text, multiple choice, dropdown, email, number, yes/no, rating	MISSING	No type enumeration, choice configuration, or question-specific schemas are defined.
Question Settings required, help text, choices	MISSING	No database fields or validation for <code>required</code> , <code>description</code> , or options.
Question Reordering	MISSING	No <code>order_index</code> column or batch reordering endpoint implemented.
Public Shareable Form URL	MISSING	Form has a <code>slug</code> , but no public route exists to serve forms to respondents by slug.
One-Question-at-a-Time Flow	MISSING	Backend provides no respondent payload formatting for progressive navigation.
Response Persistence	MISSING	No models or tables (<code>responses</code> , <code>answers</code>) to store respondent submissions.
Client & Server Validation	PARTIAL	Only Form title length is validated. Question answer validation (e.g. email regex, choice match) is missing.
Individual Response Viewing	MISSING	No endpoints to view submitted responses and answer breakdowns.
Basic Response Statistics	MISSING	No aggregation endpoints for submission counts, choice distributions, or rating averages.
Seed Data	MISSING	No script to pre-populate sample forms with all 8 question types and mock responses.
Frontend Application	MISSING	Workspace currently contains only the <code>backend/</code> folder; frontend is not yet created.

F. Architectural Problems & Risks

1. Premature Commits in Repository Layer (Unit of Work Risk)

MEDIUM RISK

`FormRepository.create()`, `update()`, and `delete()` call `self.db.commit()` directly. When composite operations are introduced (such as creating a form with questions, or saving a submission with multiple answers), committing inside individual repository methods breaks atomic transactions. If step 2 fails, step 1 cannot be rolled back.

Mitigation: Keep repository methods for staging changes, and coordinate commit/rollback boundaries in the service layer or via a transactional context manager.

2. SQLite Foreign Key Enforcement & Cascading Deletes

MEDIUM RISK

SQLite disables foreign key enforcement by default unless explicitly activated via `PRAGMA foreign_keys = ON;` upon engine connection. Without this and explicit SQLAlchemy `cascade="all, delete-orphan"`, deleting a form could leave orphaned questions and responses.

Mitigation: Add a SQLite `connect` event listener in `connection.py` to execute `PRAGMA foreign_keys = ON`, and declare cascade options on parent relationships.

3. Rigid FormUpdate Schema & Testing Dependencies

DESIGN CONSIDERATION

`FormUpdate` only permits `title: str`. Updating form status (publish / unpublish) or description will require broadening the schema to optional fields. Additionally, `requirements.txt` is missing `httpx` and `pytest`, causing `fastapi.testclient.TestClient` to fail during test execution.

G. Recommended Next Steps & Roadmap

Phase 1: Expand SQLAlchemy Models & Database Schema

DATA LAYER

Create `Question` model (with `title`, `description`, `type`, `required`, `order_index`, `options` as JSON/relationship) and `Response` / `Answer` models with cascading delete rules. Enable foreign keys in SQLite.

Phase 2: Question & Response Repositories & Services

LOGIC LAYER

Add `QuestionRepository`, `QuestionService` (handling position indexing, reordering, and type validation), and `ResponseRepository`, `ResponseService` (handling atomic submission, answer validation per type, and basic analytics aggregation). Add publish/unpublish methods to `FormService`.

Phase 3: Expose REST Endpoints

API LAYER

Implement builder routes (question CRUD & reordering), public respondent routes (`GET /api/public/forms/{slug}` and `POST /api/public/forms/{slug}/submit`), and analytics routes (`GET /api/forms/{form_id}/responses` and `/analytics`).

Phase 4: Seed Script & Verification

VERIFICATION

Write a clean seed script populating sample forms with all 8 question types and test submissions for rapid verification and evaluation.